

Summary

This topic synthesizes the fundamental concepts, practical applications, and algorithmic procedures of graph theory as presented in the "Algorithm Crash Course for GATE" series. It outlines the structural definitions of graphs, the mechanics of Depth-First Search (DFS), and the properties of spanning trees.

Graphs are essential for modern computer science, from electrical and road networks to the World Wide Web and social networking platforms. A graph (G) is formally defined by its sets of vertices (V) and edges (E). Simple graphs exclude self-loops and parallel edges, while directed graphs emphasize order and directionality.

Key takeaways:

- DFS uses a stack-based recursive approach to determine discovery and finish times for vertices.
- Spanning trees connect all nodes without cycles; a complete graph with (n) nodes contains exactly (n^{n-2}) spanning trees.
- These concepts form the foundation for advanced topics such as Greedy algorithms and Dynamic Programming.

1. Foundational Graph Definitions

A graph (G) is a mathematical structure consisting of two distinct sets:

- Vertex Set (V): A collection of nodes.
- Edge Set (E): A collection of lines connecting pairs of nodes.

1.1 Edge Characteristics

Graph Type	Edge Property	Order Relevance
Undirected	Defined by endpoints (e.g., $E1 = \{A, D\}$)	Order does not matter; $\{A, D\} = \{D, A\}$
Directed (Digraph)	Defined by direction (e.g., $E1 = W \rightarrow X$)	Order matters; $(W, X) \neq (X, W)$

1.2 Structural Variations

- Self-Loops: Edges where both endpoints are the same node.
- Parallel Edges: Multiple edges between the same pair of nodes.

- Simple Graphs: No self-loops and no parallel edges.

2. Connectivity and Adjacency

- Adjacency (Neighbors): Two nodes are adjacent if a direct edge exists between them.
- Connected Graphs: Every pair of nodes is connected by a path; otherwise, the graph is disconnected.
- Complete Graphs: Every node is adjacent to every other node.
 - Edge Calculation: For (n) nodes, the number of edges is $(\frac{n(n-1)}{2})$.

3. Practical Applications and Social Networks

- World Wide Web (WWW): Nodes are web pages, edges are hyperlinks.
- Social Networks (e.g., Facebook):
 - Nodes: User profiles.
 - Edges: Friendships (undirected).
 - Follower relationships: Directed.
 - Distance search: Identifying "friends of friends" requires exploring layers outward from the source node.

4. Graph Traversal: Depth-First Search (DFS)

Traversal is visiting every node exactly once. DFS uses a "deep" exploration strategy with recursion or an explicit stack.

4.1 Core Terminology

- Visited: Node has been first reached.
- Explored: Node has been fully explored.
- Discovery Time (d): Time step when first visited.
- Finish Time (f): Time step when exploration finishes.

4.2 DFS Mechanics

1. Start at a source node (Discovery Time = 1).
2. Move to an unvisited neighbor recursively.
3. Use recursion/stack to remember the path.
4. Upon reaching a dead end, mark finish time and backtrack.

4.3 Algorithmic Applications

DFS and BFS are used for:

- Detecting cycles.
- Checking connectivity and identifying connected components.
- Finding bridges and articulation points.
- Topological sorting.
- Determining if a graph is bipartite.

5. Spanning Trees

A spanning tree is a subgraph that connects all nodes without cycles.

5.1 Properties

- Node Consistency: Includes every node from the original graph.
- Connectivity: All nodes are connected.
- Cycle-Free: Contains no cycles.
- Edge Count: For (n) nodes, a spanning tree has exactly $(n - 1)$ edges.

5.2 Spanning Trees in Complete Graphs

The number of unique spanning trees in a complete graph with (n) nodes: $(n^{\{n-2\}})$.

Example	Calculation	Result
4-node complete graph	$(4^{\{4-2\}} = 4^2)$	16 possible spanning trees